Support Vector Machines (SVM)

CSCI-P 556

ZORAN TIGANJ

Today

- Support Vector Machines
- Slides are based on Chapter 5 from the Hands on Machine Learning Textbook.

What is SVM?

- A Support Vector Machine (SVM) is a powerful and versatile Machine Learning model.
- It is capable of performing
  - linear or nonlinear classification,
  - regression,
- SVMs are particularly well suited for classification of complex small- or medium-sized datasets.

- Slides are based on Chapter 5 from the Hands on Machine Learning Textbook.
  - We will first cover a more practical approach to SVM
  - Then we will look under the hood to see how it really works

Decision Boundaries

- Reminder about iris dataset
- This is a famous dataset that contains the sepal and petal length and width of 150 iris flowers of three different species:
  - Iris setosa
  - Iris versicolor
  - Iris virginica
- Let's try to build a classifier to detect the Iris virginica type based only on the petal width feature.

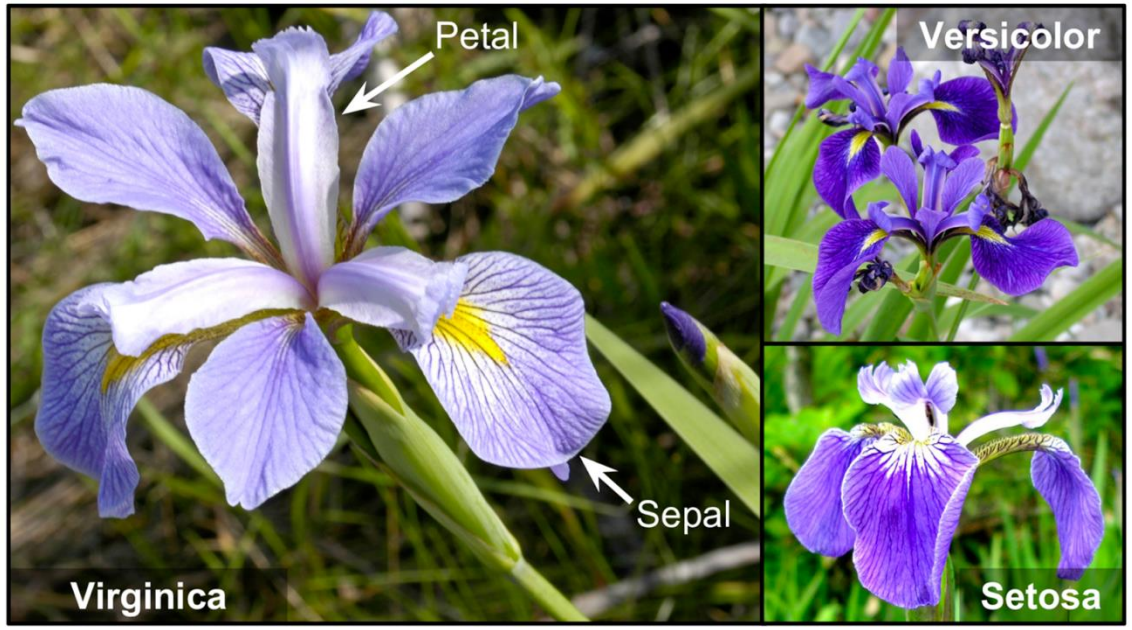
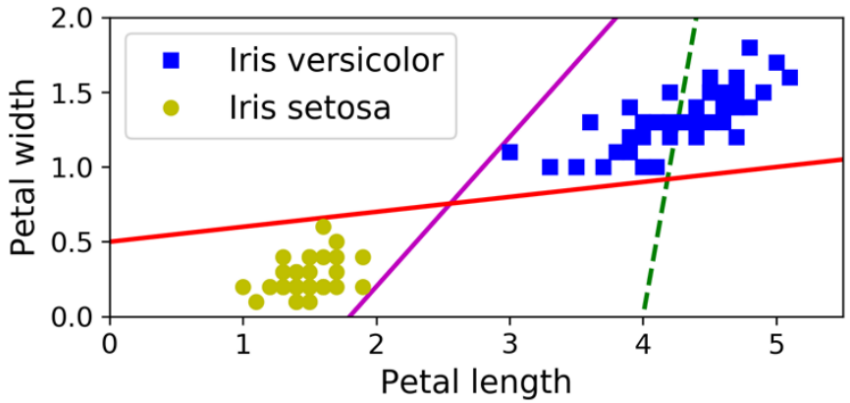
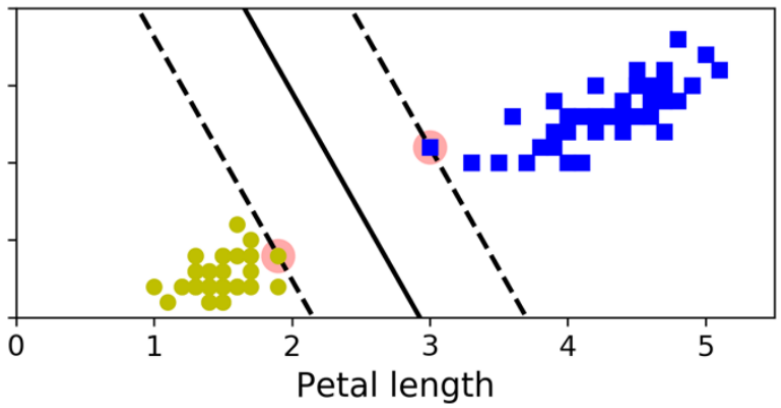


Figure 4-22. Flowers of three iris plant species[14]

Large margin classification





- Decision boundaries of three possible linear classifiers.
  - The model whose decision boundary is represented by the dashed line is so bad that it does not even separate the classes properly.
  - The other two models work perfectly on this training set, but their decision boundaries come so close to the instances that these models will probably not perform as well on new instances.
- The solid line represents the decision boundary of an SVM classifier.
  - This line not only separates the two classes but also stays as far away from the closest training instances as possible.
  - You can think of an SVM classifier as fitting the widest possible street.
  - This is called large margin classification.

Notation note

- Previously we used the convention of putting all the model parameters in one vector $\boldsymbol{\theta}$, including the bias term θ_0 and the input feature weights θ_1 to θ_n, and adding a bias input $x_0 = 1$ to all instances.
- Now we will use a convention that is more convenient (and more common) when dealing with SVMs: the bias term will be called b, and the feature weights vector will be called $\mathbf{w}$.

Decision Function and Predictions

- The linear SVM classifier model predicts the class of a new instance x by simply computing the decision function $\mathbf{w}^T \mathbf{x} + b = w_1 x_1 + \cdots + w_n x_n + b$.
- If the result is positive, the predicted class $\hat{y}$ is the positive class (1), and otherwise it is the negative class (0);

$$\hat{y} = \begin{cases} 0 & \text{if } \mathbf{w}^\intercal \mathbf{x} + b < 0, \\ 1 & \text{if } \mathbf{w}^\intercal \mathbf{x} + b \geq 0 \end{cases}$$

Decision Function and Predictions

- The decision boundary is the set of points where the decision function is equal to 0: it is the intersection of two planes, which is a straight line.

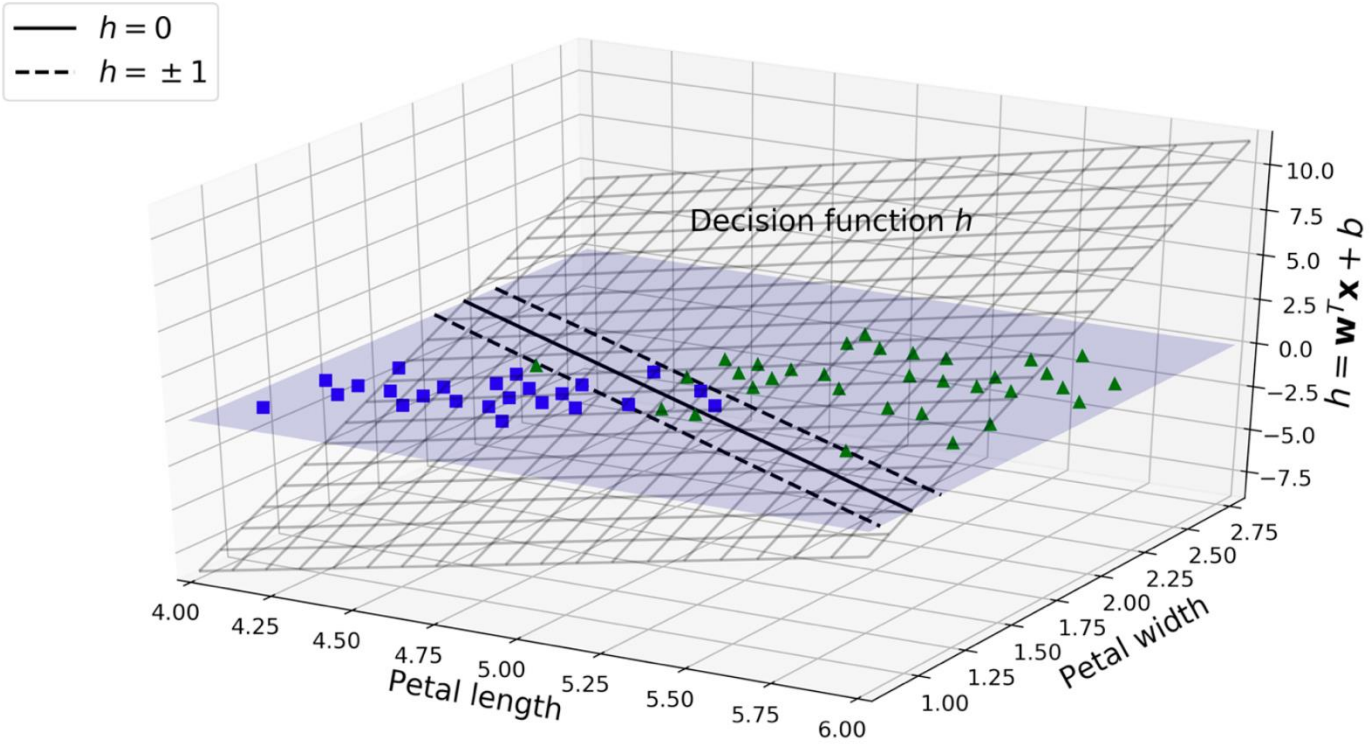
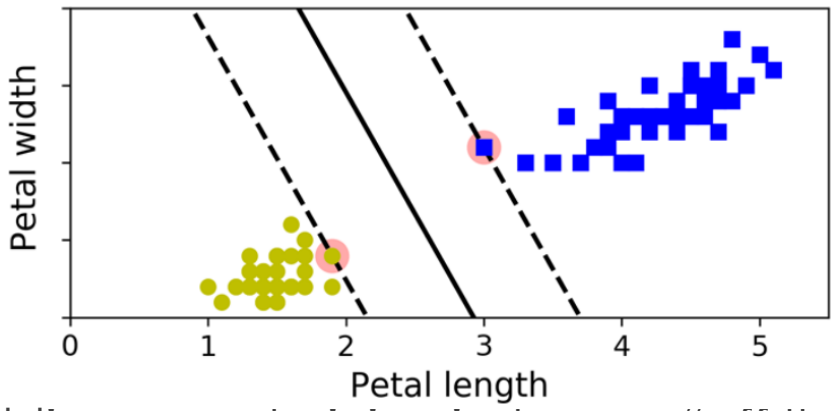


Figure 5-12. Decision function for the iris dataset

The dashed lines represent the points where the decision function is equal to 1 or –1.

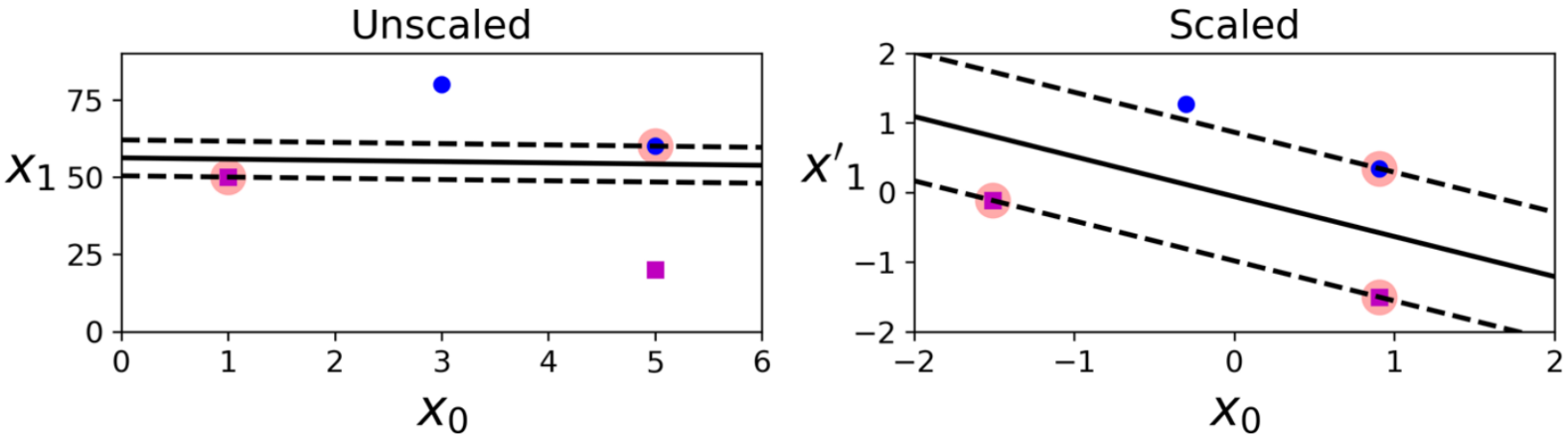
Training a linear SVM classifier means finding the values of **w** and b that make this margin as wide as possible while avoiding margin violations (hard margin) or limiting them (soft margin).

Support vectors



- Notice that adding more training instances "off the street" will not affect the decision boundary at all:
  - it is fully determined (or "supported") by the instances located on the edge of the street.
  - These instances are called the support vectors (highlighted in the figure)

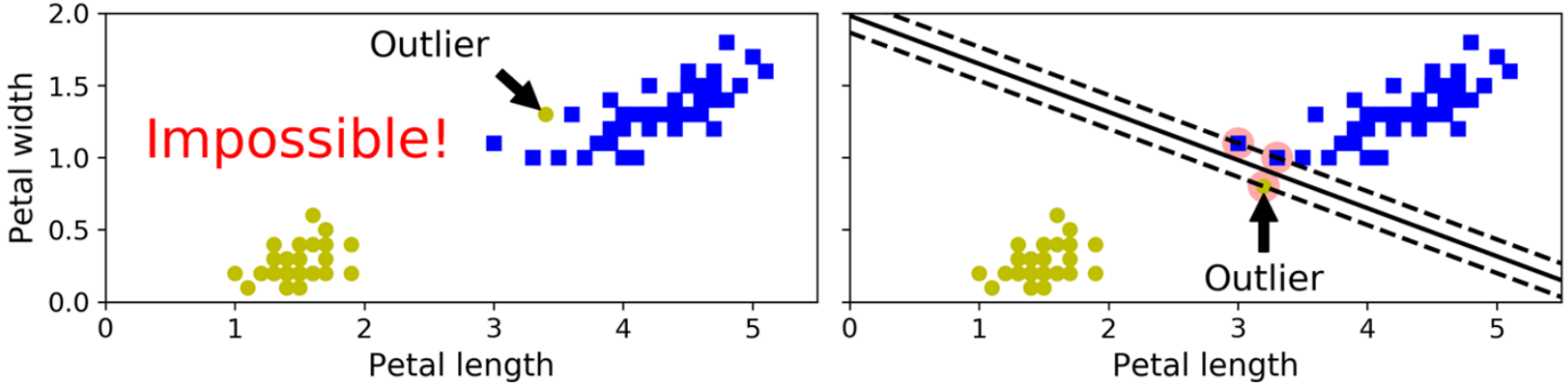
SVM and feature scaling



- SVMs are sensitive to the feature scales:
  - in the left plot, the vertical scale is much larger than the horizontal scale, so the widest possible street is close to horizontal.
  - After feature scaling (e.g., using Scikit-Learn's StandardScaler), the decision boundary in the right plot looks much better.

Soft Margin Classification

- If we strictly impose that all instances must be off the street and on the right side, this is called hard margin classification.
- There are two main issues with hard margin classification:
  - it only works if the data is linearly separable (figure on the left – not linearly separable)
  - it is sensitive to outliers (figure on the right)



To avoid these issues, use a more flexible model. The objective is to find a good balance between keeping the street as large as possible and limiting the margin violations. This is called soft margin classification .

Soft Margin Classification

- When creating an SVM model using Scikit-Learn, we can specify a number of hyperparameters.
- C is one of those hyperparameters.

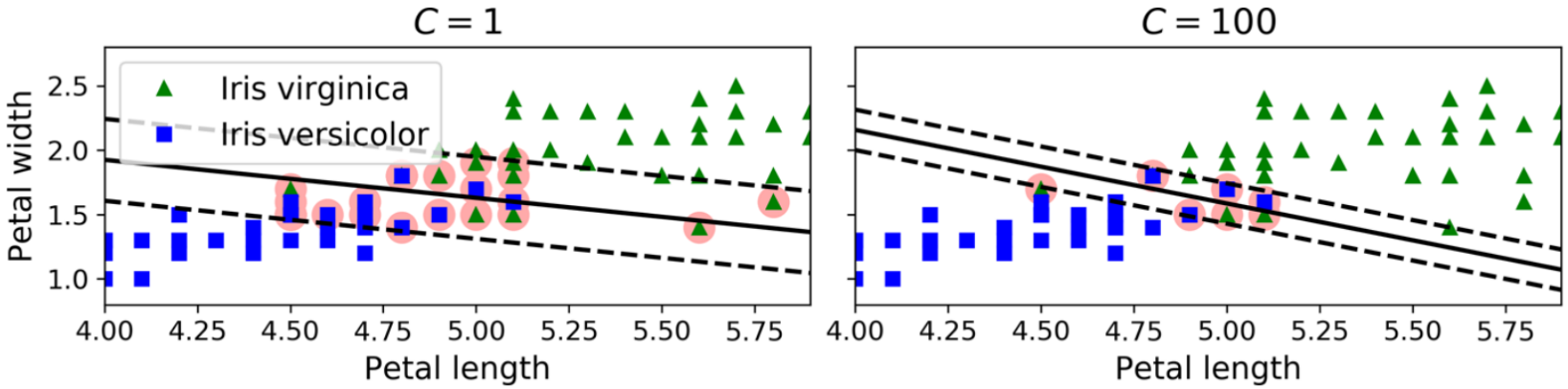


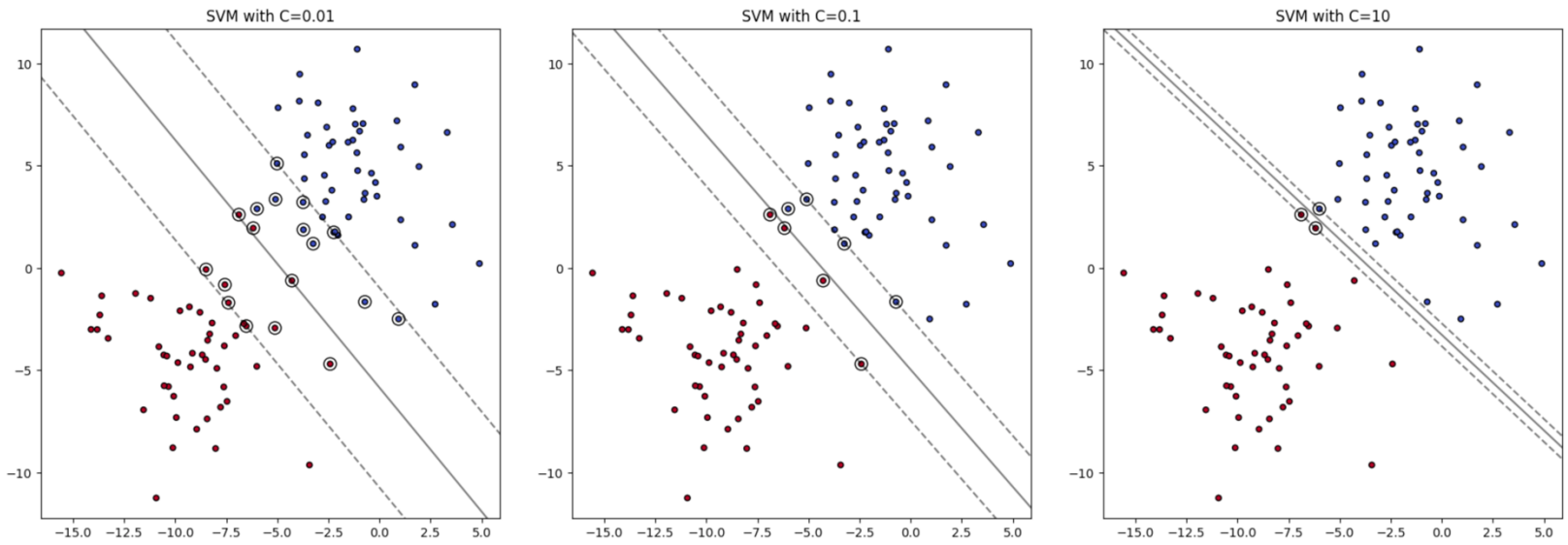
Figure 5-4. Large margin (left) versus fewer margin violations (right)

$$J(\mathbf{w}, b) = \frac{1}{2}\mathbf{w}^\intercal\mathbf{w} \quad + \quad C\sum_{i=1}^{m} max\left(0, 1 - t^{(i)}\left(\mathbf{w}^\intercal\mathbf{x}^{(i)} + b\right)\right)$$

Tip: If your SVM model is overfitting, you can try regularizing it by reducing C.

Soft Margin Classification

```
X, y = make_blobs(n_samples=100, centers=2, random_state=1, cluster_std=3.0)
C_values = [0.01, 0.1, 10]
models = [svm.SVC(kernel='linear', C=C) for C in C_values]
for model in models:
        model.fit(X, y)
```



w = -0.25, -0.20 w = -0.46, -0.40 w = -1.67, -1.70

In SVM margin (dashed lines) are defined as:

$$\mathbf{w}^T\mathbf{x} + b = 1$$

$$\mathbf{w}^T\mathbf{x} + b = -1$$

Decision boundary is

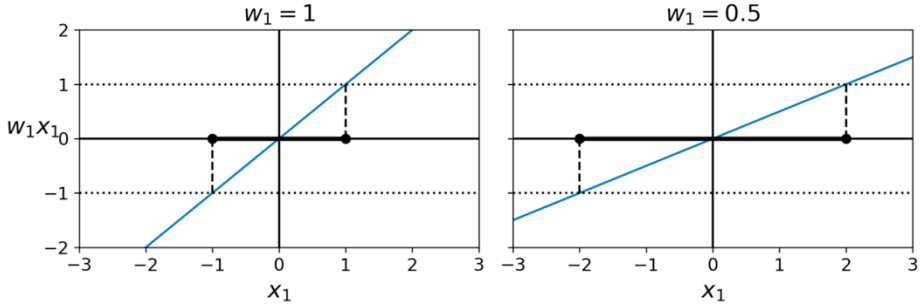
$$\mathbf{w}^T\mathbf{x} + b = 0$$

Distance between margin and decision boundary is $\frac{1}{||\boldsymbol{w}||}$

So we want to minimize **||w||** to get a large margin.

Training Objective

- Decision boundary satisfies equation $\mathbf{w}^T\mathbf{x} + b = 0$
- The slope of the decision boundary is equal to the norm of the weight vector, **‖w‖**
- Let's assume (without loss of generality) that margins (support vectors) satisfy equations $\mathbf{w}^T\mathbf{x} + b = 1$ $\mathbf{w}^T\mathbf{x} + b = -1$
- What will happen if we, say, reduce all the weights in **‖w‖** to a half of their value (if we divided them by 2)?
- The margin will increase by a factor of 2.



- So we want to minimize **‖w‖** to get a large margin.

Primal problem

- So we want to minimize $\|\mathbf{w}\|$ to get a large margin.
- If we also want to avoid any margin violations (hard margin), then we need the decision function to be greater than 1 for all positive training instances and lower than –1 f or negative training instances.
- If we define $t^{(i)} = -1$ for negative instances (if y = 0) and $t^{(i)} = 1$ for positive instances (if y = 1), then we can express this constraint as $t^{(i)}(\mathbf{w}^T \mathbf{x} + b) \geq 1$ for all instances.

$$\underset{\mathbf{w},b}{\text{minimize}} \quad \frac{1}{2}\mathbf{w}^\intercal\mathbf{w}$$

$$\text{subject to} \quad t^{(i)}\left(\mathbf{w}^\intercal\mathbf{x}^{(i)} + b\right) \geq 1 \quad \text{for } i = 1, 2, \cdots, m$$

Primal problem

- To get the soft margin objective, we need to introduce a slack variable $\zeta \geq 0$ for each instance: ζ measures how much the ith instance is allowed to violate the margin.

$$\underset{\mathbf{w},b,\zeta}{\text{minimize}} \quad \frac{1}{2}\mathbf{w}^\intercal\mathbf{w} + C\sum_{i=1}^{m}\zeta^{(i)}$$

$$\text{subject to} \quad t^{(i)}\left(\mathbf{w}^\intercal\mathbf{x}^{(i)} + b\right) \geq 1 - \zeta^{(i)} \quad \text{and} \quad \zeta^{(i)} \geq 0 \quad \text{for } i = 1, 2, \cdots, m$$

- C is a hyperparameter that defines the tradeoff between minimizing $\mathbf{w}^\intercal\mathbf{w}$ and reducing the margin violations.

Quadratic Programming

- The hard margin and soft margin problems are both convex quadratic optimization problems with linear constraints.
- Such problems are known as Quadratic Programming (QP) problems.
- Many off-the-shelf solvers are available to solve QP problems.

$$
\begin{aligned}
\underset{\mathbf{p}}{\text{Minimize}} \quad & \frac{1}{2}\mathbf{p}^\top \mathbf{H}\mathbf{p} \quad + \quad \mathbf{f}^\top \mathbf{p} \\
\text{subject to} \quad & \mathbf{A}\mathbf{p} \leq \mathbf{b} \\
\text{where} \quad & \begin{cases}
\mathbf{p} & \text{is an } n_p\text{-dimensional vector } (n_p = \text{number of parameters}), \\
\mathbf{H} & \text{is an } n_p \times n_p \text{ matrix}, \\
\mathbf{f} & \text{is an } n_p\text{-dimensional vector}, \\
\mathbf{A} & \text{is an } n_c \times n_p \text{ matrix } (n_c = \text{number of constraints}), \\
\mathbf{b} & \text{is an } n_c\text{-dimensional vector}.
\end{cases}
\end{aligned}
$$

The Dual Problem

- To use the kernel trick, we are going to look at a different constrained optimization problem: **the dual problem**
  - we won't do a derivation from primal to dual, but you can see appendix of the book, the derivation is done in Elements of AI and Machine Learning courses.

Lagrangian Formulation

$$J(\mathbf{w}, b, \alpha) = \frac{1}{2}\mathbf{w}^T\mathbf{w} - \sum_{i=1}^{N} \alpha_i [d_i(\mathbf{w}^T\mathbf{x}_i + b) - 1]$$

where

$$\alpha_i \geq 0$$

- From condition 1:

$$\frac{\partial J(\mathbf{w}, b, \alpha)}{\partial \mathbf{w}} = 0 \quad => \quad \mathbf{w} - \sum_i \alpha_i d_i \mathbf{x}_i = 0 \quad => \quad \mathbf{w} = \sum_i \alpha_i d_i \mathbf{x}_i$$

- From condition 2:

$$\frac{\partial J(\mathbf{w}, b, \alpha)}{\partial b} = 0 \quad => \quad \sum_i \alpha_i d_i = 0$$

From B555

Dual Problem

$$\underset{\alpha}{\text{minimize}}\ \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha^{(i)}\alpha^{(j)}t^{(i)}t^{(j)}\mathbf{x}^{(i)\intercal}\mathbf{x}^{(j)} \quad - \quad \sum_{i=1}^{m}\alpha^{(i)}$$

$$\text{subject to} \quad \alpha^{(i)} \geq 0 \quad \text{for } i = 1, 2, \cdots, m$$

$$\widehat{\mathbf{w}} = \sum_{i=1}^{m}\widehat{\alpha}^{(i)}t^{(i)}\mathbf{x}^{(i)}$$

$$\hat{b} = \frac{1}{n_s}\sum_{\substack{i=1 \\ \widehat{\alpha}^{(i)}>0}}^{m}\left(t^{(i)} - \widehat{\mathbf{w}}^{\intercal}\mathbf{x}^{(i)}\right)$$

Dual problem makes the kernel trick possible, while primal does not.

Nonlinear SVM Classification

- Although linear SVM classifiers are efficient and work surprisingly well in many cases, many datasets are not even close to being linearly separable.
- One approach to handling nonlinear datasets is to add more features, such as polynomial features - in some cases this can result in a linearly separable dataset.

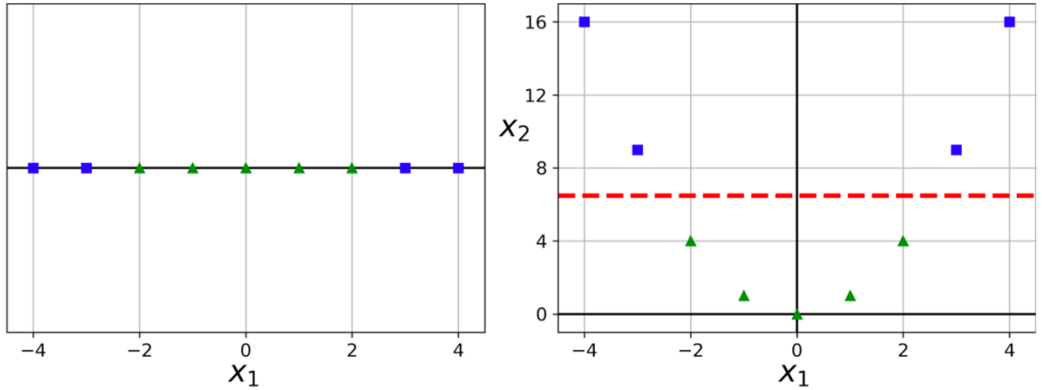


Figure 5-5. Adding features to make a dataset linearly separable

Consider the left plot: it represents a simple dataset with just one feature, x . This dataset is not linearly separable. But if you add a second feature $x_2 = (x_1)^2$, the resulting 2D dataset is perfectly linearly separable.

Kernelized SVMs

- Suppose you want to apply a second-degree polynomial transformation to a two dimensional training set, hen train a linear SVM classifier on the transformed training set.

$$\phi(\mathbf{x}) = \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} {x_1}^2 \\ \sqrt{2}\, x_1 x_2 \\ {x_2}^2 \end{pmatrix}$$

- Notice that the transformed vector is 3D instead of 2D.

Kernelized SVMs

- Now let's look at what happens to a couple of 2D vectors, **a** and **b**, if we apply this second-degree polynomial mapping and then compute the dot product of the transformed vectors

$$\phi(\mathbf{a})^\intercal \phi(\mathbf{b}) = \begin{pmatrix} {a_1}^2 \\ \sqrt{2}\, a_1 a_2 \\ {a_2}^2 \end{pmatrix}^\intercal \begin{pmatrix} {b_1}^2 \\ \sqrt{2}\, b_1 b_2 \\ {b_2}^2 \end{pmatrix} = {a_1}^2 {b_1}^2 + 2 a_1 b_1 a_2 b_2 + {a_2}^2 {b_2}^2$$

$$= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^\intercal \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = (\mathbf{a}^\intercal \mathbf{b})^2$$

If you apply the transformation ϕ to all training instances, then the dual problem will contain the dot product $\phi(\mathbf{x}^{(i)})^\intercal \phi(\mathbf{x}^{(j)})$

But if ϕ is the second-degree polynomial transformation, then you can replace this dot product of transformed vectors simply by $\left(\mathbf{x}^{(i)\intercal} \mathbf{x}^{(j)}\right)^2$

So, you don't need to transform the training instances at all!

Kernelized SVMs

- The result will be strictly the same as if you had gone through the trouble of transforming the training set then fitting a linear SVM algorithm, but this trick makes the whole process much more computationally efficient.
- The function $K(\mathbf{a},\mathbf{b}) = (\mathbf{a}^T\mathbf{b})^2$ is a second-degree polynomial kernel.
- In Machine Learning, a kernel is a function capable of computing the dot product $\varphi(\mathbf{a})^T\,\varphi(\mathbf{b})$, based only on the original vectors **a** and **b**, without having to compute (or even to know about) the transformation φ.

Examples of Kernels:

$$\text{Linear:} \quad K(\mathbf{a},\mathbf{b}) = \mathbf{a}^T\mathbf{b}$$

$$\text{Polynomial:} \quad K(\mathbf{a},\mathbf{b}) = (\gamma\mathbf{a}^T\mathbf{b} + r)^d$$

$$\text{Gaussian RBF:} \quad K(\mathbf{a},\mathbf{b}) = \exp\left(-\gamma\|\mathbf{a}-\mathbf{b}\|^2\right)$$

$$\text{Sigmoid:} \quad K(\mathbf{a},\mathbf{b}) = \tanh(\gamma\mathbf{a}^T\mathbf{b} + r)$$

Kernelized SVMs

MERCER'S THEOREM

According to *Mercer's theorem*, if a function $K(\mathbf{a}, \mathbf{b})$ respects a few mathematical conditions called *Mercer's conditions* (e.g., K must be continuous and symmetric in its arguments so that $K(\mathbf{a}, \mathbf{b}) = K(\mathbf{b}, \mathbf{a})$, etc.), then there exists a function ϕ that maps $\mathbf{a}$ and $\mathbf{b}$ into another space (possibly with much higher dimensions) such that $K(\mathbf{a}, \mathbf{b}) = \phi(\mathbf{a})^\intercal \phi(\mathbf{b})$. You can use K as a kernel because you know ϕ exists, even if you don't know what ϕ is. In the case of the Gaussian RBF kernel, it can be shown that ϕ maps each training instance to an infinite-dimensional space, so it's a good thing you don't need to actually perform the mapping!

Note that some frequently used kernels (such as the sigmoid kernel) don't respect all of Mercer's conditions, yet they generally work well in practice.

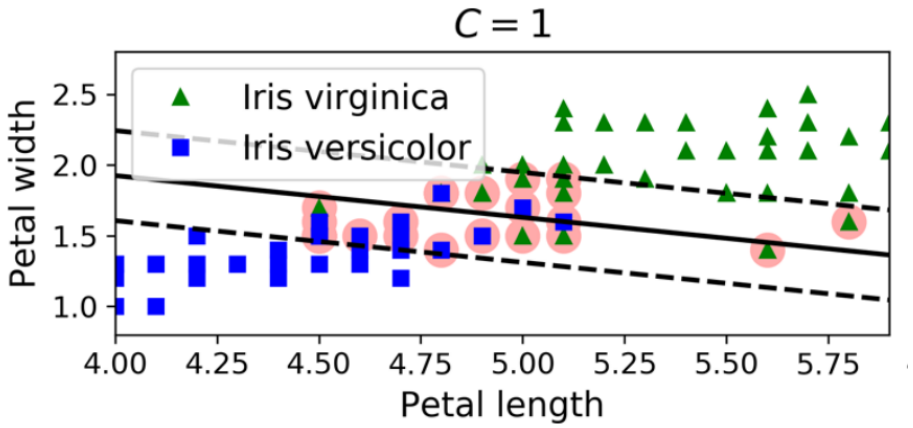
Let's try this on Iris dataset

```
import numpy as np
from sklearn import datasets
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

iris = datasets.load_iris()
X = iris["data"][:, (2, 3)]  # petal length, petal width
y = (iris["target"] == 2).astype(np.float64)  # Iris virginica

svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("linear_svc", LinearSVC(C=1, loss="hinge")),
    ])

svm_clf.fit(X, y)
```



Nonlinear SVM Classification

- Although linear SVM classifiers are efficient and work surprisingly well in many cases, many datasets are not even close to being linearly separable.
- One approach to handling nonlinear datasets is to add more features, such as polynomial features - in some cases this can result in a linearly separable dataset.

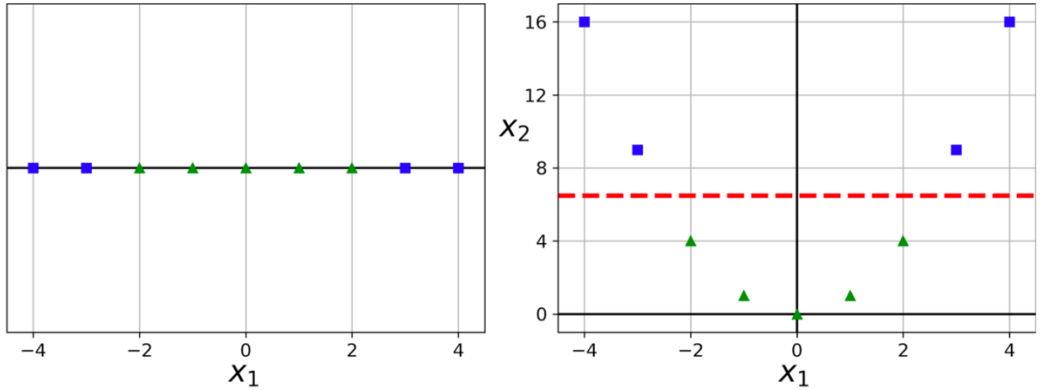


Figure 5-5. Adding features to make a dataset linearly separable

Consider the left plot: it represents a simple dataset with just one feature, x . This dataset is not linearly separable. But if you add a second feature $x_2 = (x_1)^2$, the resulting 2D dataset is perfectly linearly separable.

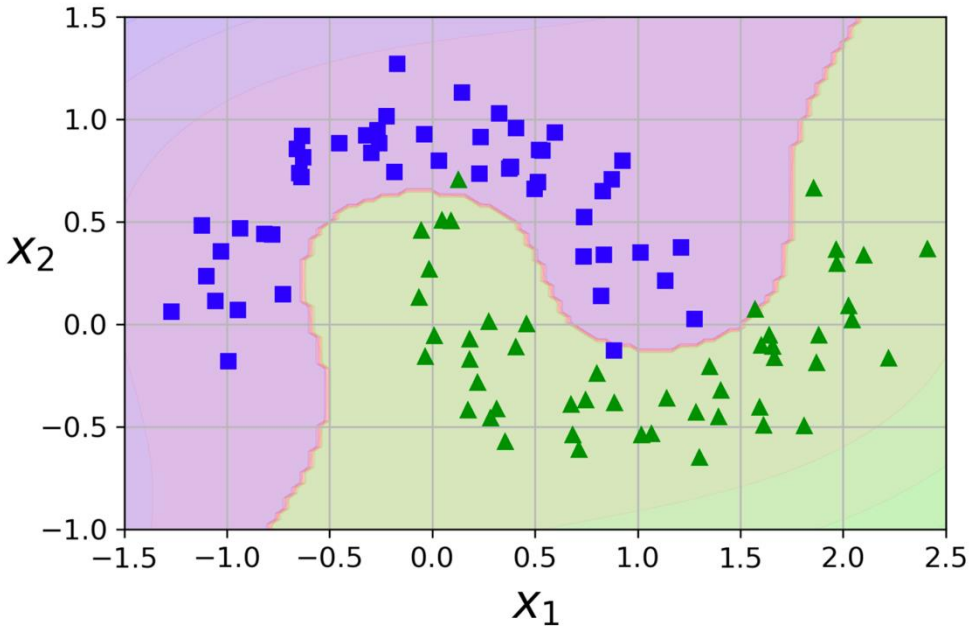
Let's test this on the moons dataset

- Moons dataset: a toy dataset for binary classification in which the data points are shaped as two interleaving half circles.

```
from sklearn.datasets import make_moons
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures

X, y = make_moons(n_samples=100, noise=0.15)
polynomial_svm_clf = Pipeline([
        ("poly_features", PolynomialFeatures(degree=3)),
        ("scaler", StandardScaler()),
        ("svm_clf", LinearSVC(C=10, loss="hinge"))
    ])

polynomial_svm_clf.fit(X, y)
```



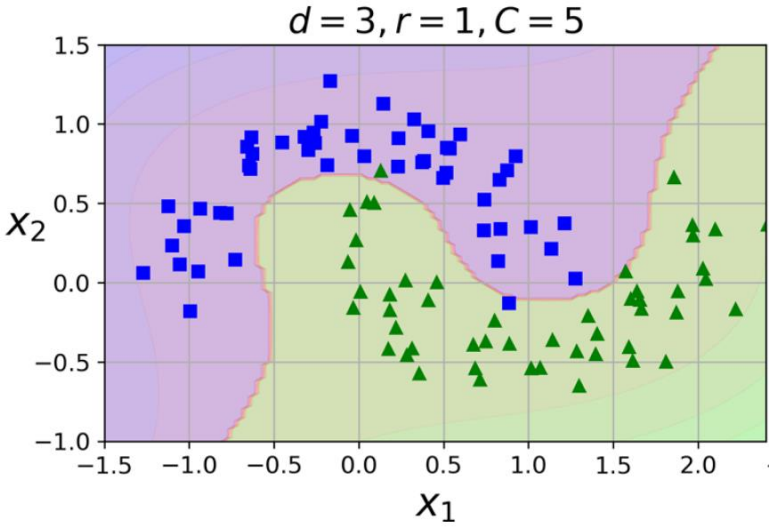
Polynomial Kernel

- Adding polynomial features is simple to implement and can work great with all sorts of Machine Learning algorithms (not just SVMs).
- That said, at a low polynomial degree, this method cannot deal with very complex datasets, and with a high polynomial degree it creates a huge number of features, making the model too slow.
- Fortunately, when using SVMs you can apply an almost miraculous mathematical technique called the kernel trick.
- **The kernel trick makes it possible to get the same result as if you had added many polynomial features, even with very high-degree polynomials, without actually having to add them.**
- So there is no combinatorial explosion of the number of features because you don't actually add any features.

Polynomial Kernel

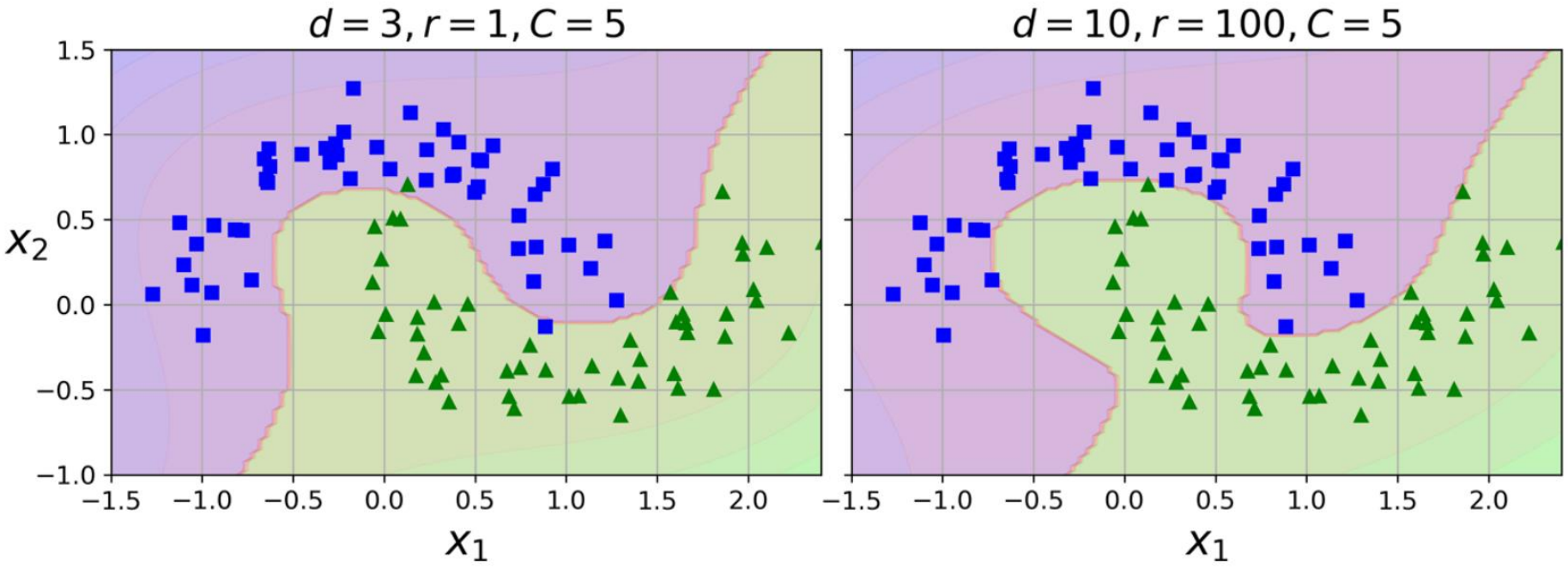
- This trick is implemented by the SVC class. Let's test it on the moons dataset.
- This code trains an SVM classifier using a third-degree polynomial kernel:

```
from sklearn.svm import SVC
poly_kernel_svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("svm_clf", SVC(kernel="poly", degree=3, coef0=1, C=5))
    ])
poly_kernel_svm_clf.fit(X, y)
```



The hyperparameter coef0 controls how much the model is influenced by high-degree polynomials versus low-degree polynomials.

Polynomial Kernel

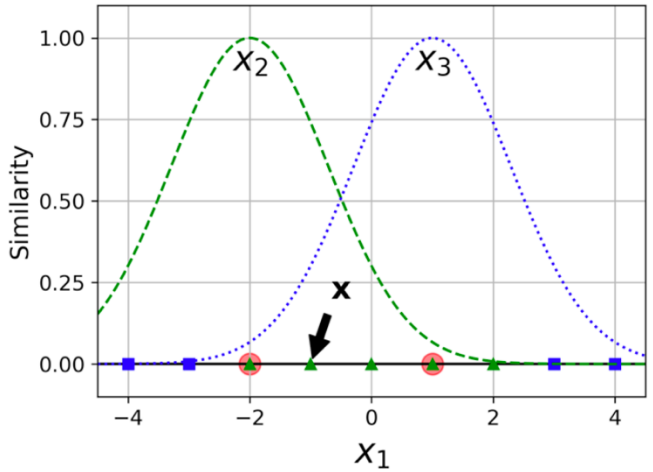


If your model is overfitting, you might want to reduce the polynomial degree. Conversely, if it is underfitting, you can try increasing it.

Similarity Features

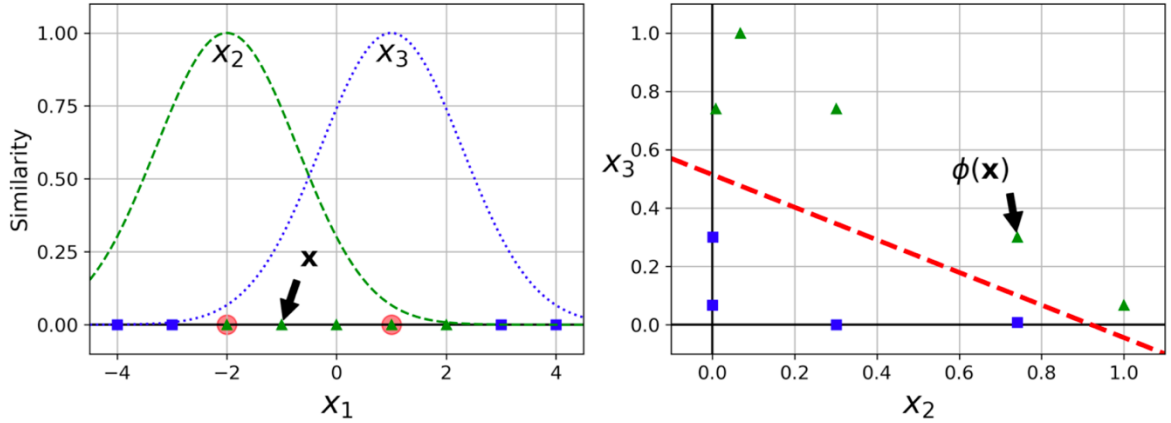
- Another technique to tackle nonlinear problems is to add features computed using a similarity function, which measures how much each instance resembles a particular landmark.
  - let's take the 1D dataset discussed earlier and add two landmarks to it at $x_1 = -2$ and $x_1 = 1$.
  - Next, let's define the similarity function to be the Gaussian Radial Basis Function (RBF) with $\gamma = 0.3$

$$\phi_\gamma(\mathbf{x}, \ell) = \exp\left(-\gamma\|\mathbf{x} - \ell\|^2\right)$$



Similarity Features

- Now we are ready to compute the new features.
  - For example, let's look at the instance $x_1 = -1$: it is located at a distance of 1 from the first landmark and 2 from the second landmark.
  - Therefore its new features are
    - $x_2 = \exp(-0.3 \times 1^2) \approx 0.74$ and
    - $x_3 = \exp(0.3 \times 2^2) \approx 0.30$.



The plot on the right shows the transformed dataset (dropping the original features). It is now linearly separable.

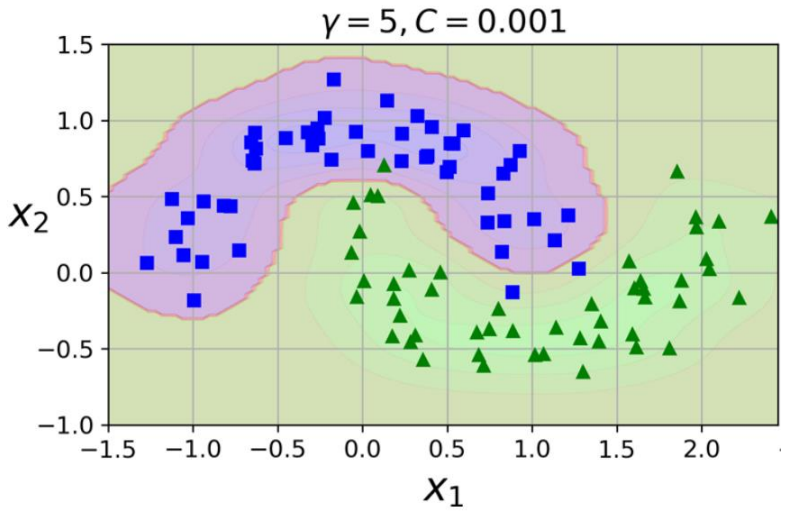
Similarity Features

- How to select the landmarks?
  - The simplest approach is to create a landmark at the location of each and every instance in the dataset.
  - Doing that creates many dimensions and thus increases the chances that the transformed training set will be linearly separable.
  - Downside: If your training set is very large, you end up with an equally large number of features.

Gaussian RBF Kernel

- Just like the polynomial features method, the similarity features method can be useful with any Machine Learning algorithm, but it may be computationally expensive to compute all the additional features, especially on large training sets.
- Once again the kernel trick does its SVM magic, making it possible to obtain a similar result as if you had added many similarity features.
- Let's try the SVC class with the Gaussian RBF kernel

```
rbf_kernel_svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("svm_clf", SVC(kernel="rbf", gamma=5, C=0.001))
    ])
rbf_kernel_svm_clf.fit(X, y)
```



Gaussian RBF Kernel

- Increasing γ makes the bell-shaped curve narrower:
  - As a result, each instance's range of influence is smaller: the decision boundary ends up being more irregular, wiggling around individual instances.
- Conversely, a small γ value makes the bell-shaped curve wider:
  - instances have a larger range of influence, and the decision boundary ends up smoother.
- So γ acts like a regularization hyperparameter:
  - if your model is overfitting, you should reduce it;
  - if it is underfitting, you should increase it (similar to the C hyperparameter).

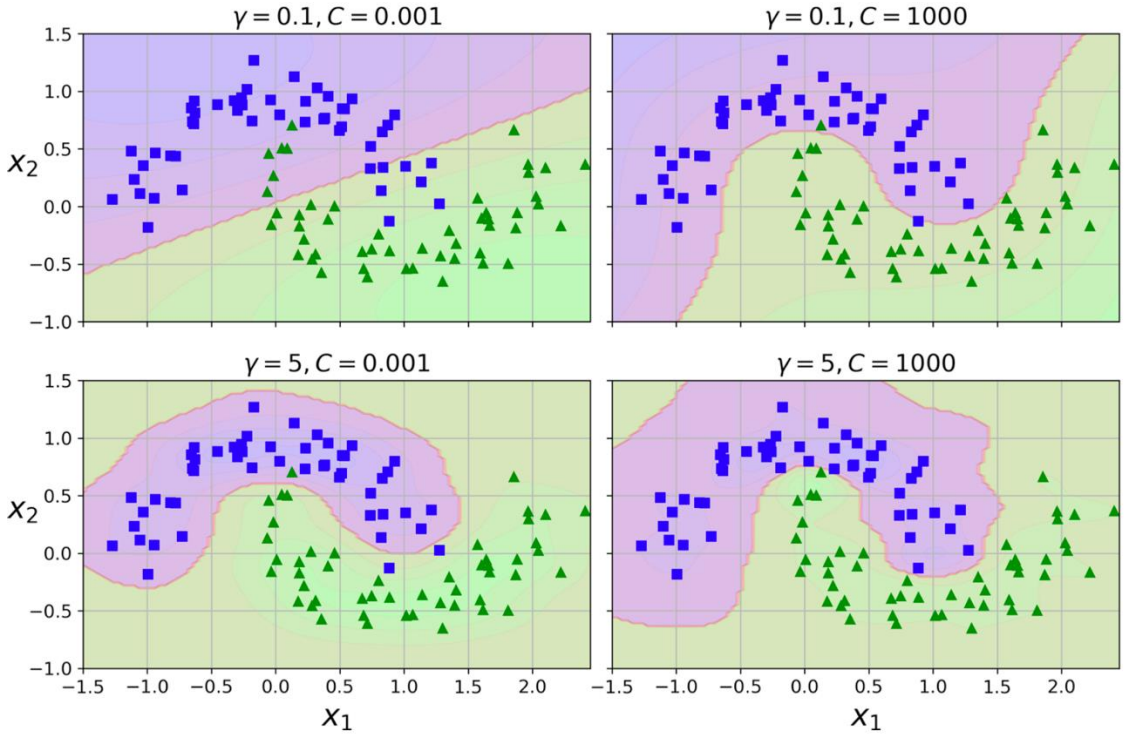


Figure 5-9. SVM classifiers using an RBF kernel

Other kernels

- Other kernels exist but are used much more rarely.
- Some kernels are specialized for specific data structures:
  - String kernels are sometimes used when classifying text documents or DNA sequences (e.g., using the string subsequence kernel or kernels based on the Levenshtein distance).
  - String kernels can be intuitively understood as functions measuring the similarity of pairs of strings: the more similar two strings a and b are, the higher the value of a string kernel K(a, b) will be.

Which kernel to choose

- With so many kernels to choose from, how can you decide which one to use?
  - As a rule of thumb, you should always try the linear kernel first, especially if the training set is very large or if it has plenty of features.
  - If the training set is not too large, you should also try the Gaussian RBF kernel; it works well in most cases.
  - Then if you have spare time and computing power, you can experiment with a few other kernels, using cross-validation and grid search.

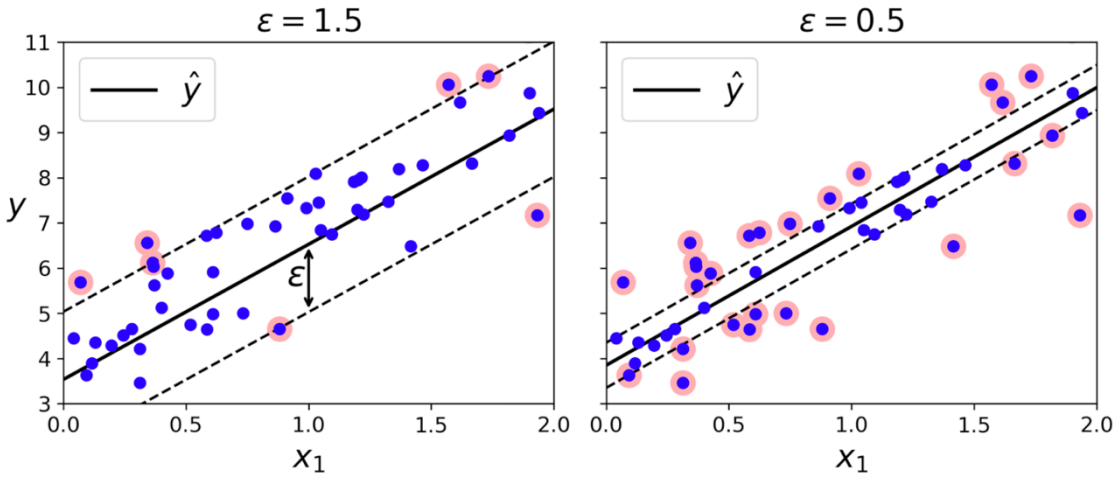
Computational Complexity

- The LinearSVC class is based on the liblinear library, which implements an optimized algorithm for linear SVMs. It does not support the kernel trick, but it scales almost linearly with the number of training instances and the number of features.
- The SVC class is based on the libsvm library, which implements an algorithm that supports the kernel trick. It gets dreadfully slow when the number of training instances gets large (e.g., hundreds of thousands of instances). This algorithm is perfect for complex small or medium-sized training sets.

Class	Time complexity	Out-of-core support	Scaling required	Kernel trick
`LinearSVC`	$O(m \times n)$	No	Yes	No
`SGDClassifier`	$O(m \times n)$	Yes	Yes	No
`SVC`	$O(m^2 \times n)$ to $O(m^3 \times n)$	No	Yes	Yes

SVM Regression

- The SVM algorithm is versatile: not only does it support linear and nonlinear classification, but it also supports linear and nonlinear regression.
- To use SVMs for regression instead of classification, the trick is to reverse the objective:
  - instead of trying to fit the largest possible street between two classes while limiting margin violations, SVM Regression tries to fit as many instances as possible on the street while limiting margin violations (i.e., instances off the street).
  - The width of the street is controlled by a hyperparameter, ε.

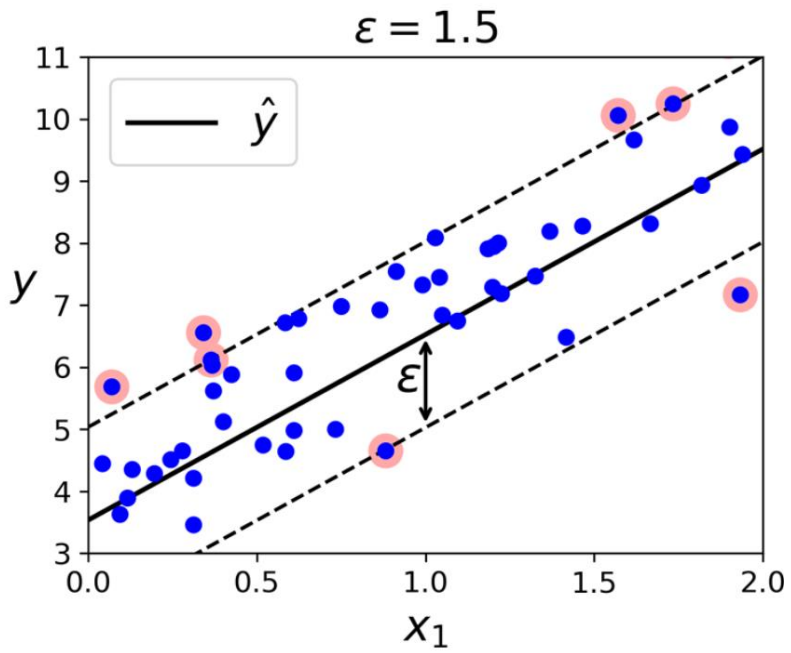


SVM Regression

- You can use Scikit-Learn's LinearSVR class to perform linear SVM Regression.

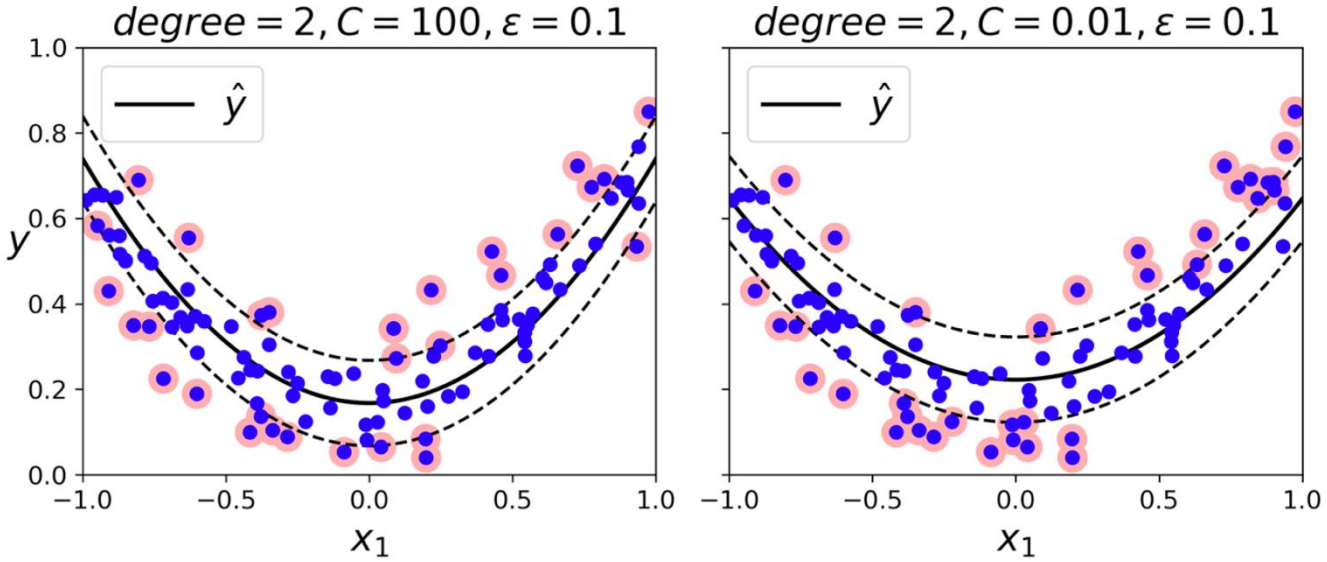
```
from sklearn.svm import LinearSVR

svm_reg = LinearSVR(epsilon=1.5)
svm_reg.fit(X, y)
```



SVM Regression

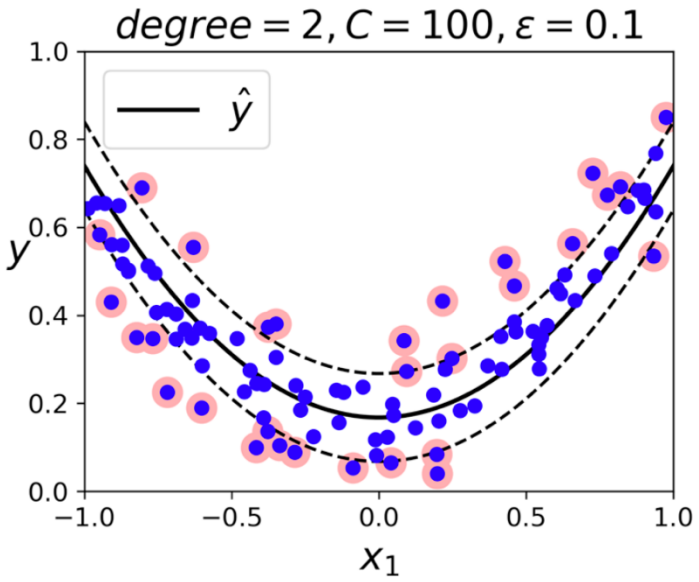
- To tackle nonlinear regression tasks, you can use a kernelized SVM model.
- Figure shows SVM Regression on a random quadratic training set, using a second-degree polynomial kernel.
- There is little regularization in the left plot (i.e., a large C value), and much more regularization in the right plot (i.e., a small C value).



SVM Regression

```
from sklearn.svm import SVR

svm_poly_reg = SVR(kernel="poly", degree=2, C=100, epsilon=0.1)
svm_poly_reg.fit(X, y)
```



Next time

- Decision trees, from Chapter 6, *Hands on machine learning textbook*